

Regular Types in the Bernoulli Model: When Equality Isn't Equal

Alexander Towell
atowell@siue.edu

July 29, 2025

Abstract

We explore the fundamental tension between regular types—which assume perfect observation of equality—and Bernoulli types, where we can only observe latent equality through a noisy channel. Regular types require that equality be reflexive, symmetric, and transitive, but these axioms assume we can perfectly observe whether two values are equal. In reality, equality comparison is itself a computation that observes the latent mathematical fact of equality. Bernoulli types make this explicit: when we write `a == b`, we are not accessing the latent truth but rather observing it through potentially faulty comparison operations. This shift from assuming perfect equality to observing approximate equality enables new programming paradigms for distributed systems (where different nodes may observe different equalities), privacy-preserving computation (where equality is deliberately obscured), and fault-tolerant algorithms (where hardware errors affect observations). We present a type system that distinguishes latent equality from observed equality, enabling safe integration of Bernoulli types with traditional programming.

1 Introduction

Alexander Stepanov's concept of regular types forms the foundation of generic programming [6]. A regular type supports:

- **Copy construction:** `T a(b)`; creates `a` as a copy of `b`
- **Assignment:** `a = b`; makes `a` equal to `b`
- **Equality:** `a == b` returns `true` iff `a` and `b` are equal
- **Destruction:** Objects can be destroyed

These operations must satisfy semantic constraints. Most critically, equality must be:

- **Reflexive:** `a == a`
- **Symmetric:** `a == b` implies `b == a`
- **Transitive:** `a == b` and `b == c` implies `a == c`

Bernoulli types reveal that these constraints assume perfect observation of equality. In reality:

- **Latent:** Two values are either mathematically equal or not
- **Observed:** We can only observe this equality through computation

When `operator==` returns `bernoulli<bool>`, it acknowledges this fundamental gap:

```
bernoulli<int> a(42);
assert(a == a); // Observing latent equality; observation might be incorrect!
```

This paper argues that recognizing the latent/observed distinction in equality is not a deficiency but a more honest model of computation.

2 Background: Regular Types

2.1 The Regular Type Concept

Stepanov defines regular types as those that behave like built-in types:

Definition 2.1 (Regular Type). *A type T is regular if it supports:*

1. *Default construction:* $T\ a;$
2. *Copy construction:* $T\ a(b);$
3. *Move construction:* $T\ a(std::move(b));$
4. *Copy assignment:* $a = b;$
5. *Move assignment:* $a = std::move(b);$
6. *Equality:* $a == b;$
7. *Inequality:* $a != b;$

with natural semantics.

2.2 Why Regularity Matters

Regular types enable generic programming:

```
template<typename T>
void sort(std::vector<T>& v) {
    // Requires T to be regular
    // Uses copy, assignment, and comparison
}
```

The STL assumes regularity for types stored in containers. This assumption enables efficient algorithms and predictable behavior.

2.3 The Equality Problem

Equality is fundamental to regularity. It enables:

- Container lookups: `std::find`, `std::map`
- Algorithm correctness: `std::unique`, `std::set`
- Reasoning about code: Equational reasoning

But what if equality itself is uncertain?

3 Bernoulli Types and Regularity

3.1 Probabilistic Equality

For Bernoulli types, equality returns a Bernoulli boolean:

```
template<typename T>
class bernoulli {
    T value;
    double error_rate;
public:
    bernoulli<bool> operator==(const bernoulli<T>& other) const {
        // Returns true with probability related to actual equality
        return sample_bernoulli_bool(compute_equality_probability(value, other.value));
    }
};
```

3.2 The Latent/Observed Gap in Equality Axioms

3.2.1 Reflexivity: Observing Self-Equality

Latent reflexivity always holds: a value is mathematically equal to itself. But observation introduces error:

```
bernoulli<int> x(42, 0.1); // 10% error rate
auto result = (x == x);    // Observing latent self-equality
```

The gap between latent and observed arises from:

- **Measurement uncertainty:** Physical sensors observing the same value twice
- **Transmission errors:** Network corrupting equality checks
- **Privacy mechanisms:** Deliberately noisy observations for security
- **Hardware faults:** Bit flips during comparison

Mathematically:

$$\mathbb{P}\left[\widetilde{(x = x)} = \text{true} \mid \text{latent } x = x\right] = 1 - \epsilon \quad (1)$$

3.2.2 Symmetry: Independent Observations

Latent symmetry holds: if $a = b$ then $b = a$. However, each observation is independent:

$$\mathbb{P}\left[\widetilde{(a = b)} \mid \text{latent } a = b\right] = \mathbb{P}\left[\widetilde{(b = a)} \mid \text{latent } b = a\right] \quad (2)$$

But individual observations may differ:

```
if (a == b) {          // First observation of latent equality
    assert(b == a);    // Second independent observation - might differ!
}
```

This reflects reality: asking "is A equal to B?" twice may yield different answers due to measurement noise, timing, or observer differences.

3.2.3 Transitivity: Compounding Observation Errors

Transitivity suffers most severely. Even when latent transitivity holds ($a = b \wedge b = c \implies a = c$), observations compound errors:

```
bernoulli<int> a(1), b(1), c(1); // Latently all equal
if (a == b && b == c) {          // Two successful observations
    assert(a == c); // Third independent observation - often fails!
}
```

For error rate ϵ , the probability of observing transitivity when it latently holds:

$$\mathbb{P}\left[\widetilde{(a = c)} \mid (\widetilde{(a = b)}) \wedge (\widetilde{(b = c)}) \wedge \text{latent } a = b = c \mid =\right] 1 - \epsilon \quad (3)$$

The key insight: we're not observing "transitivity" but rather three independent observations of latent equalities.

3.3 Cascading Effects

The probabilistic nature of equality affects all derived operations:

- **Containers:** `std::set<bernoulli<T>>` may contain "duplicates"
- **Algorithms:** `std::sort` may not produce a total order
- **Invariants:** Class invariants become probabilistic

4 New Programming Paradigms

4.1 Probabilistic Contracts

Traditional contracts assume deterministic predicates:

```
void insert(std::set<T>& s, const T& value) {
    // Postcondition: s.count(value) == 1
}
```

With Bernoulli types, contracts become probabilistic:

```
void insert(bernoulli_set<T>& s, const T& value) {
    // Postcondition: P[value in s] >= 1 - epsilon
}
```

4.2 Eventually-Consistent Data Structures

Bernoulli types naturally model eventually-consistent systems:

Example 4.1 (Distributed Set). *In a distributed system with network partitions:*

```

class distributed_set {
    bernoulli_set<T> local_view;

    bernoulli<bool> contains(const T& value) {
        // Returns probabilistic result based on:
        // - Local information
        // - Network reliability
        // - Synchronization lag
    }
};

```

4.3 Byzantine Fault Tolerance

Bernoulli types can model Byzantine failures:

```

template<typename T>
class byzantine_value {
    std::vector<bernoulli<T>> replicas;

    bernoulli<T> read() {
        // Majority voting with probabilistic agreement
        return majority_vote(replicas);
    }
};

```

5 Practical Considerations

5.1 When to Use Bernoulli Types

Bernoulli types are appropriate when:

- **Uncertainty is inherent:** Sensor data, network communication
- **Approximation is acceptable:** Large-scale analytics, caching
- **Privacy is required:** Differential privacy, secure computation
- **Fault tolerance is critical:** Distributed systems, blockchain

5.2 Mixing Deterministic and Probabilistic Code

Safe integration requires careful boundaries:

```

// Convert Bernoulli to deterministic with threshold
template<typename T>
std::optional<T> to_deterministic(const bernoulli<T>& b, double confidence) {
    if (b.confidence() >= confidence) {
        return b.expected_value();
    }
    return std::nullopt;
}

```

```

}

// Lift deterministic to Bernoulli
template<typename T>
bernoulli<T> to_bernoulli(const T& value) {
    return bernoulli<T>(value, 0.0); // Zero error rate
}

```

5.3 Type System Extensions

We propose type system extensions for safety:

```

// Type-level error bounds
template<typename T, double MaxError>
class bounded_bernoulli {
    static_assert(MaxError >= 0.0 && MaxError <= 1.0);
    // ...
};

// Concepts for Bernoulli-aware algorithms
template<typename T>
concept BernoulliComparable = requires(T a, T b) {
    { a == b } -> std::convertible_to<bernoulli<bool>>;
};

```

6 Case Studies

6.1 Distributed Consensus

Traditional consensus algorithms assume reliable equality checks. With Bernoulli types, we model realistic networks:

Example 6.1 (Probabilistic Paxos). `class probabilistic_paxos {`

```

    struct proposal {
        int ballot;
        bernoulli<value_type> value;
    };

    bernoulli<bool> is_majority(const std::vector<vote>& votes) {
        // Account for probabilistic equality in vote counting
        int agrees = 0, total = 0;
        for (const auto& v : votes) {
            auto match = (v.value == proposed_value);
            agrees += match.probability_true();
            total++;
        }
        return bernoulli<bool>(agrees > total/2.0);
    }
};

```

6.2 Privacy-Preserving Equality

Differential privacy requires adding noise to queries:

Example 6.2 (Private Database). `class private_database {`
 double epsilon; // Privacy parameter

 bernoulli<bool> equals(const record& a, const record& b) {
 bool actual = (a.id == b.id);
 // Add Laplace noise for differential privacy
 double noise = laplace_noise(1.0 / epsilon);
 double probability = sigmoid(actual ? 1.0 + noise : noise);
 return bernoulli<bool>::from_probability(probability);
 }
};

6.3 Approximate Deduplication

Large-scale systems often need approximate deduplication:

Example 6.3 (Content-Addressable Storage). `class approximate_cas {`
 struct hash_result {
 bernoulli<hash_type> hash;
 double collision_probability;
 };

 void store(const data& content) {
 auto h = hash(content);
 if (probably_unique(h)) {
 backing_store[h.expected()] = content;
 } else {
 // Handle probable duplicate
 merge_or_create_new(h, content);
 }
 }
};

7 Implementation Strategies

7.1 Efficient Bernoulli Booleans

```
class bernoulli_bool {  
    bool value;  
    float confidence; // Stored as fixed-point for efficiency  
  
public:  
    bernoulli_bool operator&&(const bernoulli_bool& other) const {  
        // Fuzzy logic operations  
        return bernoulli_bool{
```

```

        value && other.value,
        confidence * other.confidence
    };
}

explicit operator bool() const {
    // Threshold conversion for legacy code
    return confidence > 0.5 ? value : sample_bernoulli(confidence);
}
};

```

7.2 Container Adaptations

Standard containers need adaptation for Bernoulli types:

```

template<typename T>
class bernoulli_set {
    using comparator = std::function<bernoulli<bool>(const T&, const T&)>;

    struct node {
        T value;
        double insertion_confidence;
        std::vector<std::pair<T, double>> possible_duplicates;
    };

    void insert(const T& value) {
        // Probabilistic insertion with duplicate tracking
        for (auto& n : nodes) {
            auto eq = compare(value, n.value);
            if (eq.probability_true() > duplicate_threshold) {
                n.possible_duplicates.push_back({value, eq.probability_true()});
                return;
            }
        }
        nodes.push_back({value, 1.0, {}});
    }
};

```

8 Related Work

8.1 Probabilistic Programming

Languages like Church [2] and Stan [1] support probabilistic computation but don't address the regular type concept directly.

8.2 Fuzzy Logic and Computing

Fuzzy set theory [7] provides mathematical foundations for approximate membership, which Bernoulli types extend to general computation.

8.3 Eventual Consistency

CRDTs [5] achieve eventual consistency through commutative operations, while Bernoulli types model the uncertainty during convergence.

8.4 Approximate Computing

Work on approximate hardware [3] and quality-of-service programming [4] shares goals but focuses on different abstraction levels.

9 Future Directions

9.1 Language Integration

Future programming languages could provide first-class support:

```
// Hypothetical syntax
probable<int> x = 42 +/- 0.1; // Value with uncertainty
if probably (x == 42) {      // Probabilistic branching
    // Execute with high probability
} else maybe {               // Lower probability branch
    // Handle uncertainty
}
```

9.2 Verification Tools

Static analysis for probabilistic properties:

- Probability of invariant violations
- Error propagation analysis
- Convergence guarantees

9.3 Hardware Support

Probabilistic computing primitives in hardware:

- Native Bernoulli boolean operations
- Approximate equality instructions
- Stochastic arithmetic units

10 Philosophical Implications

10.1 The Nature of Computational Truth

Bernoulli types reveal that computation doesn't access truth directly—it observes it. The latent/observed distinction is not a limitation but a fundamental aspect of physical computation. Even "deterministic" systems are simply those where observation error is negligible.

10.2 Identity Through Observation

The apparent violation of reflexivity ($x == x$ may be false) doesn't mean identity is broken—it means we can only observe identity, not access it directly. Consider:

- **Latent:** A value is always identical to itself
- **Observed:** Each observation of self-identity may differ
- **Reality:** Different parts of a distributed system may observe different identities

10.3 From Determinism to Observation

Traditional programming assumes perfect observation—that our code directly manipulates latent mathematical objects. Bernoulli types acknowledge that all computation is mediated by observation:

- Physical hardware observes mathematical operations
- Networks observe messages through noisy channels
- Sensors observe the physical world imperfectly
- Even "pure" functions observe their inputs through the lens of representation

This isn't the end of determinism but its proper contextualization: determinism is the special case where observation is perfect.

11 Conclusions

Regular types assume perfect observation of equality—that when we write `a == b`, we access the latent mathematical truth. Bernoulli types acknowledge that equality comparison is itself an observation through a potentially noisy channel. This shift from assumed perfect observation to explicit approximate observation has profound implications:

- **Honesty:** Acknowledges that all computation observes latent truth imperfectly
- **Robustness:** Systems expecting observation errors handle real-world failures gracefully
- **Scalability:** Accepting approximate observations enables better scaling
- **Privacy:** The gap between latent and observed can protect sensitive information
- **Distributed systems:** Different nodes may observe different equalities—a feature, not a bug

The apparent violation of regularity axioms reveals a deeper truth: those axioms assume we can perfectly observe mathematical relationships. In reality, every equality check, every comparison, every computation is an observation of latent mathematical truth through the lens of physical implementation.

By making the latent/observed distinction explicit, Bernoulli types don't break programming—they make it more honest about the nature of computation. As systems grow in scale and complexity, embracing the fundamental gap between what is mathematically true and what we can observe becomes not just useful but necessary.

References

- [1] Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76(1), 2017.
- [2] Noah Goodman, Vikash Mansinghka, Daniel Roy, Keith Bonawitz, and Joshua B Tenenbaum. Church: a language for generative models. In *Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence*, pages 220–229, 2008.
- [3] Jie Han and Michael Orshansky. Approximate computing: An emerging paradigm for energy-efficient design. In *2013 18th IEEE European Test Symposium (ETS)*, pages 1–6. IEEE, 2013.
- [4] Henry Hoffmann, Stelios Sidiroglou, Michael Carbin, Sasa Misailovic, Anant Agarwal, and Martin Rinard. Dynamic knobs for responsive power-aware computing. In *ACM SIGPLAN Notices*, volume 46, pages 199–212. ACM, 2011.
- [5] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *International symposium on stabilization, safety, and security of distributed systems*, pages 386–400. Springer, 2011.
- [6] Alexander Stepanov and Daniel E Rose. *From mathematics to generic programming*. Pearson Education, 2014.
- [7] Lotfi A Zadeh. Fuzzy sets. *Information and control*, 8(3):338–353, 1965.